
Table of Contents

.....	1
qst a.1:	1
question b.2	2
qst c.5:	3
qst c.6	4
qst c.7	5
qst d.8	6
qst d.9	6
qst d.10	7
2 Deconvolution qst a.11	8
qst a.12	9
qst a.13	10
qst b.14	12
qst b.15	13
qst b.16	16
qst b.17	17
qst c.18	18

```
% 1. Construction du jeu de données
%      a. Simulation du signal d'entrée

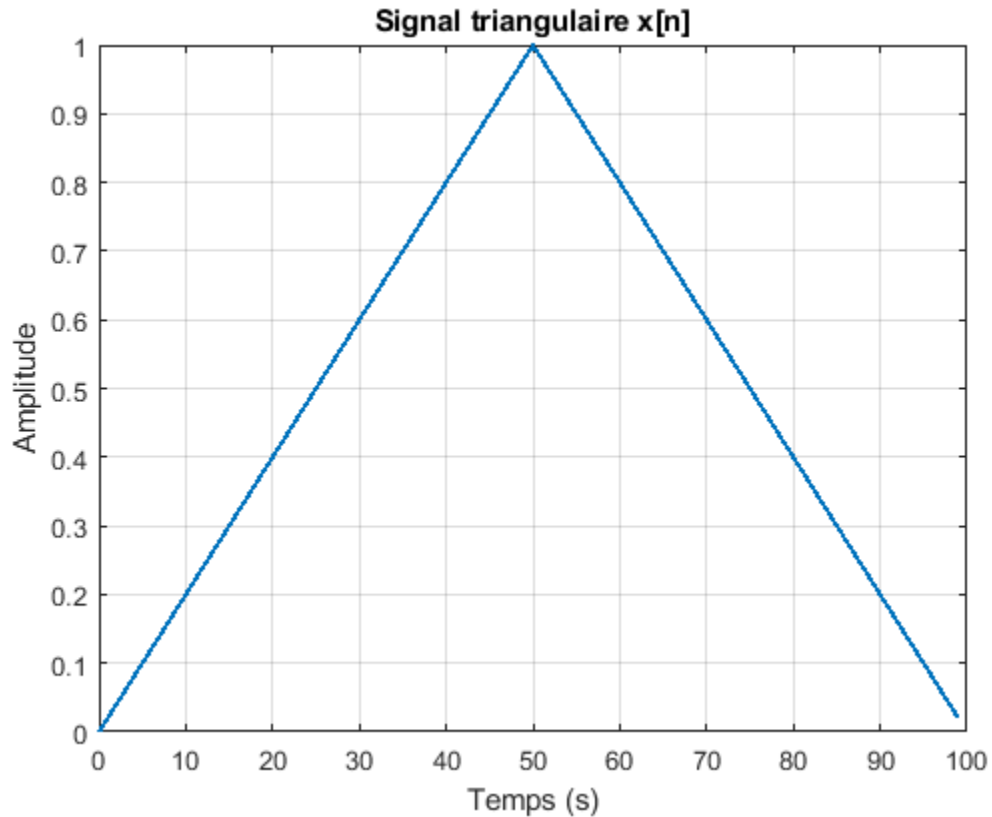
fe=1; % frequence d' echantillonnage (Hz)
x=[]; % Les échantillons du signal d'entrée
h=[]; % la réponse impulsionnelle du filtre
y=[]; % signal mesuré en sortie
b=[]; % incertitudes additives

L = 100; % dure d' observation du signal d' entree ( secondes )
Te = 1/ fe ; % periode d' echantillonnage ( secondes )
Nx = L/ Te ; % nombre d' echantillons du signal x
nx = 0: Nx -1; % indice temporel ( entier )
tx = nx * Te ; % base de temps ( secondes )
```

qst a.1:

```
% Construction du signal triangulaire
x1 = (2 / L) * tx(tx <= L/2); % Partie croissante
x2 = (-2 / L) * tx(tx > L/2) + 2; % Partie décroissante
x = [x1, x2]; % Signal complet

% Affichage
figure;
plot(tx, x, 'LineWidth', 1.5);
xlabel('Temps (s)');
ylabel('Amplitude');
title('Signal triangulaire x[n]');
grid on;
```



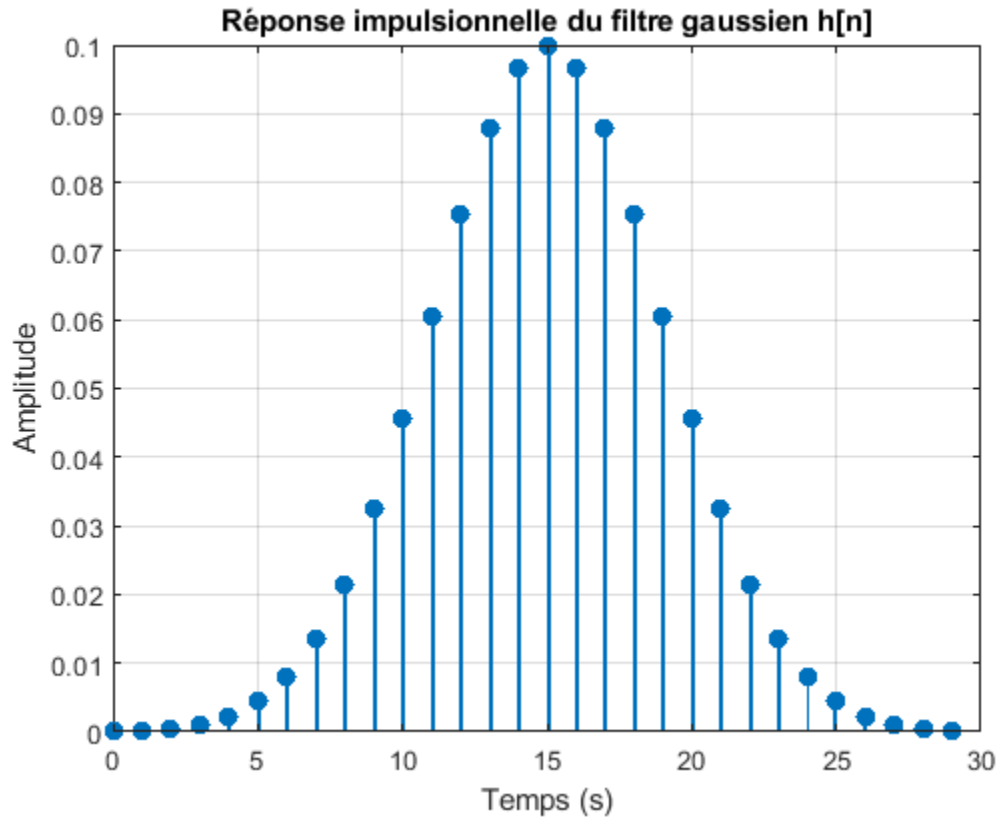
question b.2

```
% Paramètres du filtre
m = 15;           % Moyenne
sigma = 4;        % Écart-type
Nh = 30;          % Horizon de la réponse impulsionnelle (secondes)

% Base temporelle pour h[n]
n = 0:Nh-1;       % Indices temporels (causal, t >= 0)
t = n * Te;       % Base de temps (secondes)

% Calcul de h[n] (réponse impulsionnelle du filtre gaussien)
h = (1 / (sigma * sqrt(2 * pi))) * exp(-((t - m).^2) / (2 * sigma^2));

% Représentation graphique
figure;
stem(t, h, 'filled', 'LineWidth', 1.5);
xlabel('Temps (s)');
ylabel('Amplitude');
title('Réponse impulsionnelle du filtre gaussien h[n]');
grid on;
```

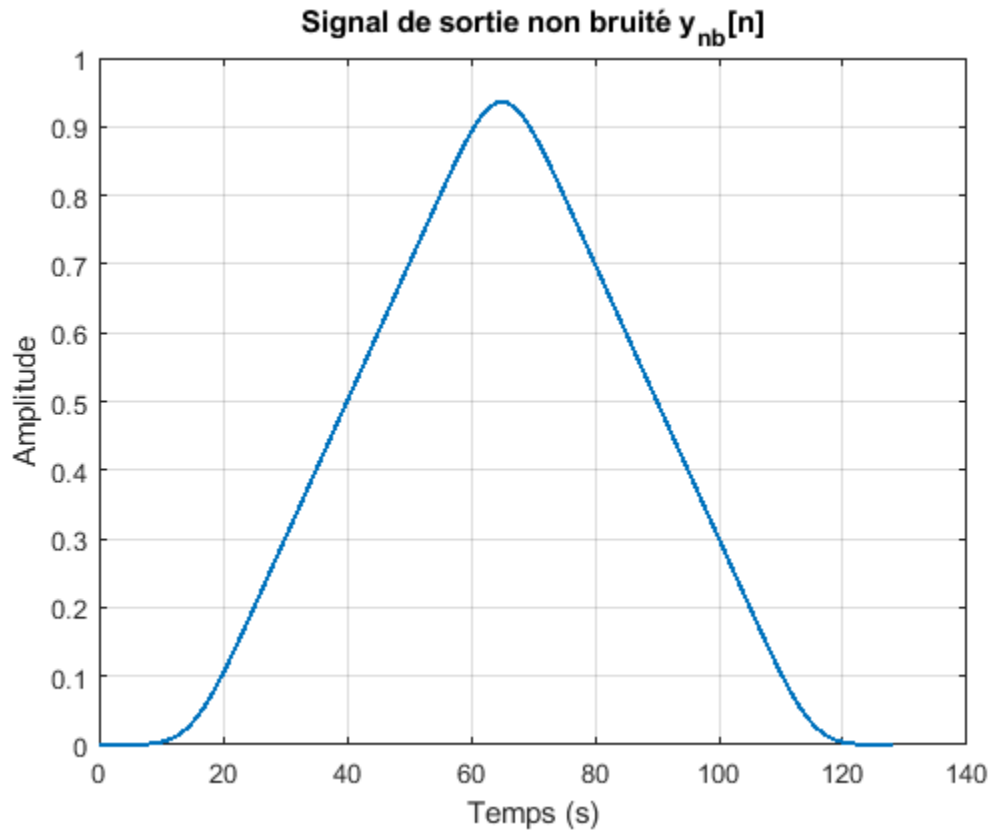


qst c.5:

```
% Calcul de la convolution pour obtenir le signal de sortie non bruité
ynb = conv(x, h); % Convolution discrète
```

```
% Base de temps pour y[n]
Ny = length(ynb); % Longueur du signal de sortie
ty = (0:Ny-1) * Te; % Base de temps correspondante
```

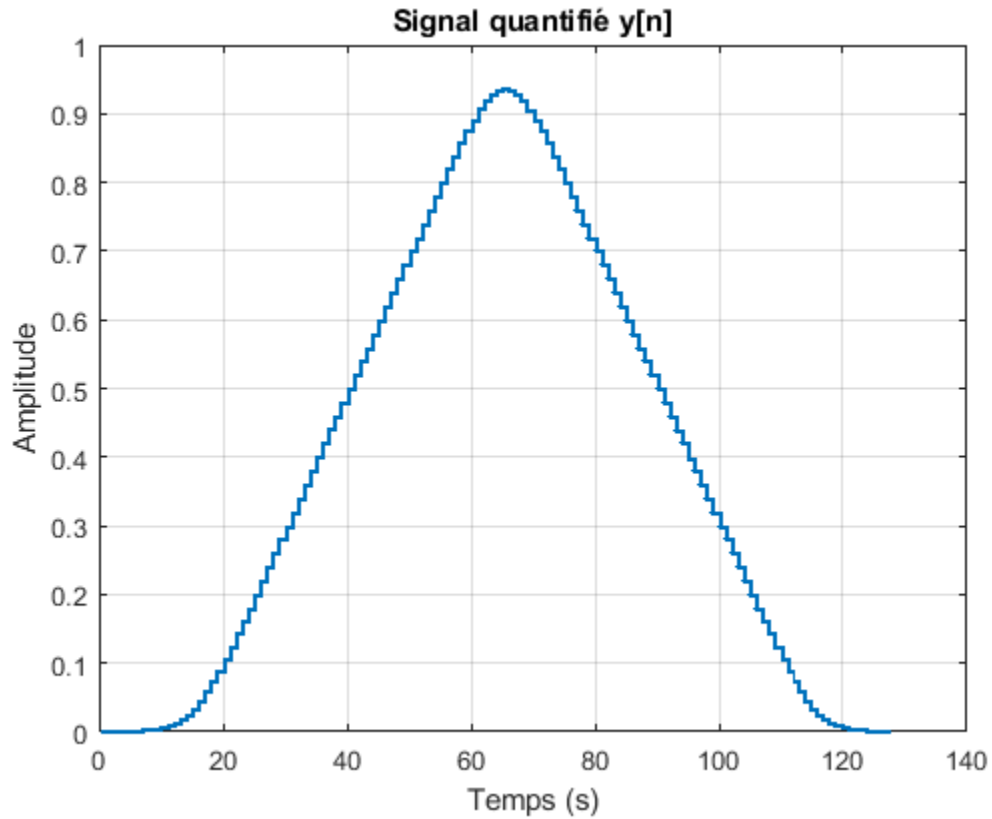
```
% Représentation graphique du signal de sortie non bruité
figure;
plot(ty, ynb, 'LineWidth', 1.5);
xlabel('Temps (s)');
ylabel('Amplitude');
title('Signal de sortie non bruité y_{nb}[n]');
grid on;
```



qst c.6

```
% Simulation de la quantification
quant_step = 2^10; % Pas de quantification
y_quant = round(y_nb * quant_step) / quant_step; % Signal quantifié

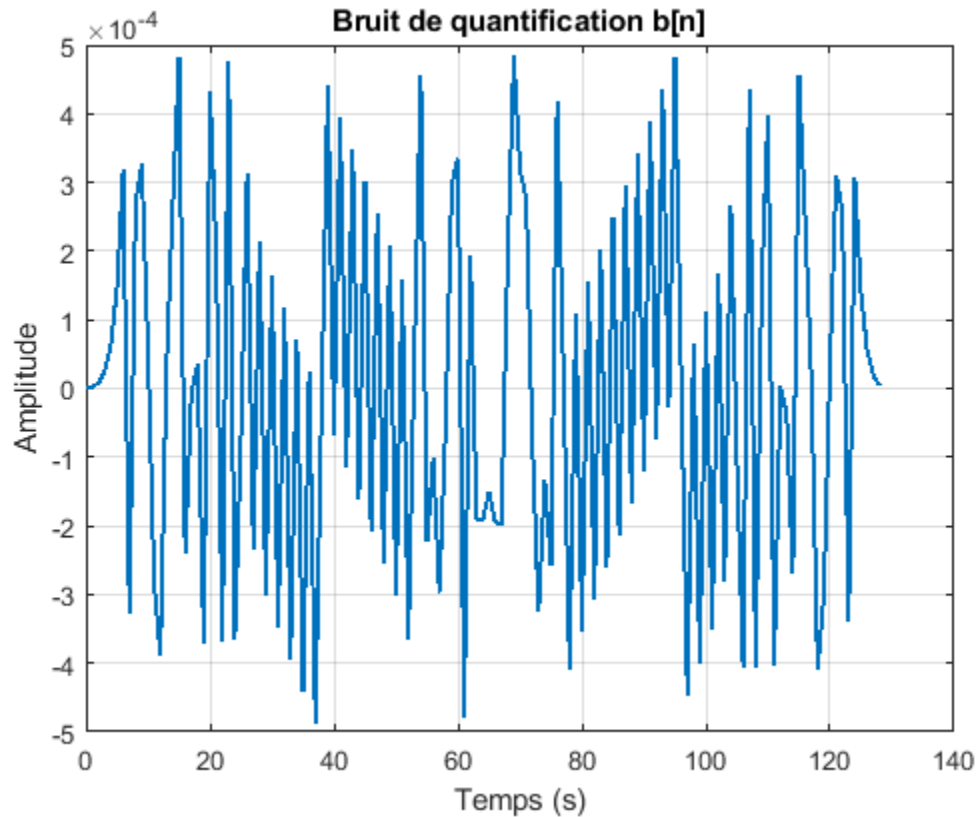
% Représentation graphique avec stairs
figure;
stairs(ty, y_quant, 'LineWidth', 1.5); % Utilisation de stairs pour un
affichage discret
xlabel('Temps (s)');
ylabel('Amplitude');
title('Signal quantifié y[n]');
grid on;
```



qst c.7

```
% Calcul du bruit de quantification
b = ynb - y_quant;

% Représentation graphique du bruit de quantification
figure;
plot(ty, b, 'LineWidth', 1.5); % Représentation du bruit
xlabel('Temps (s)');
ylabel('Amplitude');
title('Bruit de quantification b[n]');
grid on;
```



qst d.8

```
% Longueurs des signaux d'entrée et de la réponse impulsionnelle
Nx = length(x);    % Nombre d'échantillons du signal d'entrée x[n]
Nh = length(h);    % Nombre d'échantillons de la réponse impulsionnelle h[n]

% Calcul de la longueur du signal de sortie non bruité
Ny = Nx + Nh - 1;

% Affichage
fprintf('La dimension de y_{nb}[n] est : Ny= %d\n', Ny);

La dimension de y_{nb}[n] est : Ny= 129
```

qst d.9

```
h = h(:);

% Dimensions
Nx = length(x);    % Nombre d'échantillons du signal d'entrée x[n]
Nh = length(h);    % Nombre d'échantillons de la réponse impulsionnelle h[n]
Ny = Nx + Nh - 1; % Longueur du signal de sortie y[n]

% Construction de la première colonne de H
prem_col = [h; zeros(Ny - Nh, 1)]; % h suivi de zéros
```

```
% Construction de la première ligne de H
prem_lig = [h(1), zeros(1, Nx - 1)]; % h[0] suivi de zéros

% Construction de la matrice de convolution H
H = toeplitz(prem_col, prem_lig);

% Vérification des dimensions de H
fprintf('Dimensions de H : %d x %d\n', size(H, 1), size(H, 2));

Dimensions de H : 129 x 100
```

qst d.10

```
x=x(:);
% Calcul de la sortie non bruitée par opération matricielle
ynb = H * x;

% Application de l'erreur de troncature
ynb_truncated = ynb(1:Nx); % Tronquer pour correspondre à la longueur de x[n]

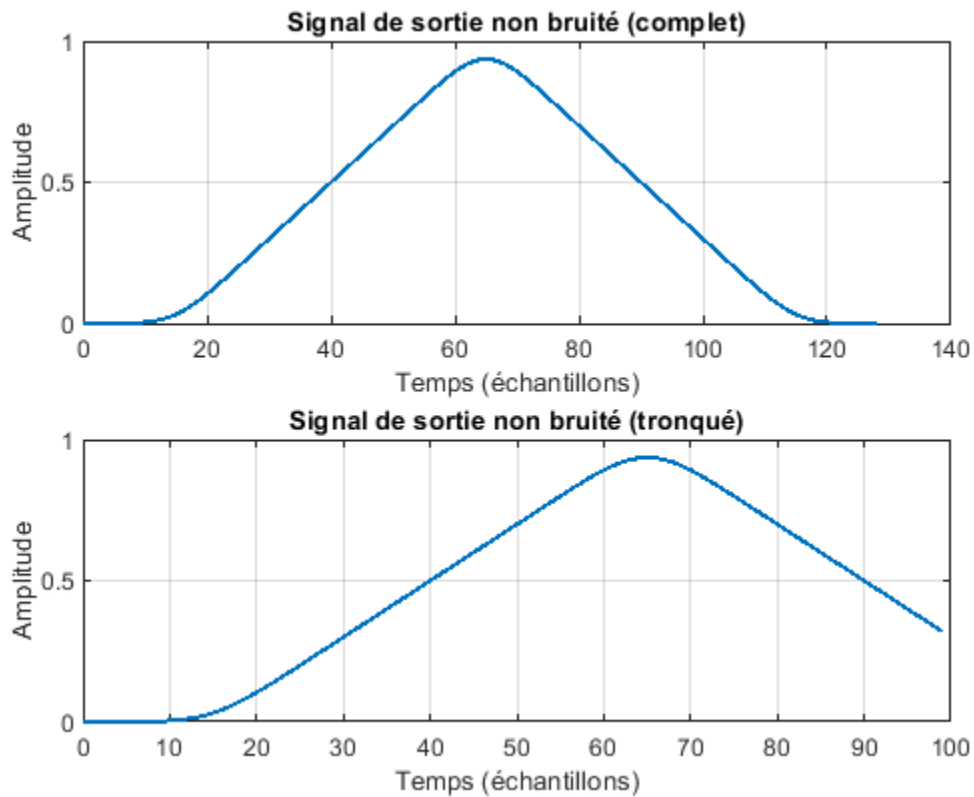
% Affichage des résultats
figure;

% Signal non bruité (avant troncature)
subplot(2, 1, 1);
plot(0:length(ynb)-1, ynb, 'LineWidth', 1.5);
xlabel('Temps (échantillons)');
ylabel('Amplitude');
title('Signal de sortie non bruité (complet)');
grid on;

% Signal non bruité tronqué
subplot(2, 1, 2);
plot(0:Nx-1, ynb_truncated, 'LineWidth', 1.5);
xlabel('Temps (échantillons)');
ylabel('Amplitude');
title('Signal de sortie non bruité (tronqué)');
grid on;

% Vérification des dimensions
fprintf('Longueur de y_{nb}[n] (complet) : %d\n', length(ynb));
fprintf('Longueur de y_{nb}[n] (tronqué) : %d\n', length(ynb_truncated));

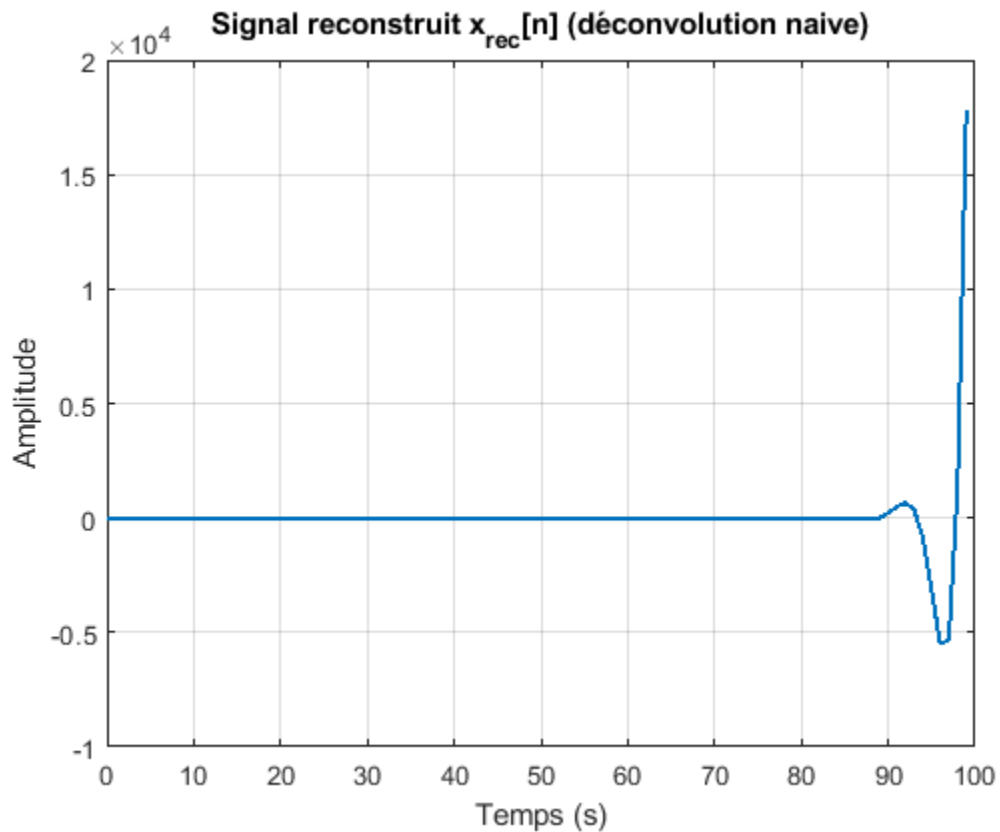
Longueur de y_{nb}[n] (complet) : 129
Longueur de y_{nb}[n] (tronqué) : 100
```



2 Deconvolution qst a.11

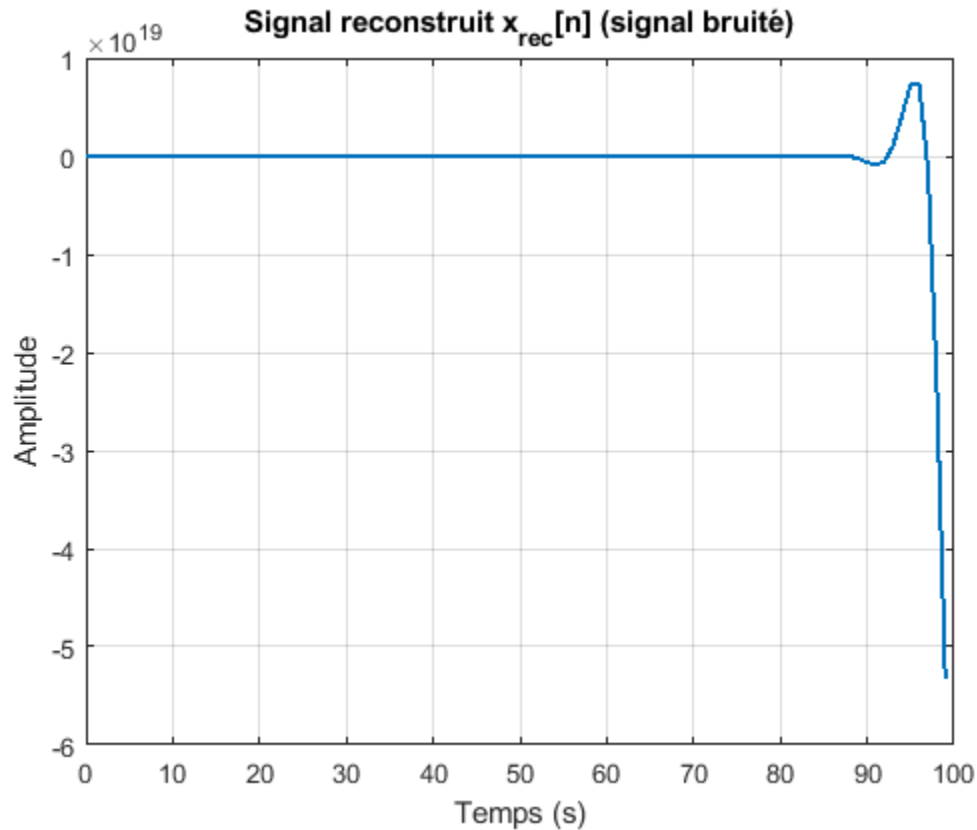
Déconvolution "naïve" dans le cas idéal (sortie non bruitée)

```
[x_rec, reste] = deconv(ynb, h);  
  
% Base de temps pour le signal reconstruit  
tx_rec = (0:length(x_rec)-1) * Te;  
  
% Représentation graphique du signal reconstruit  
figure;  
plot(tx_rec, x_rec, 'LineWidth', 1.5);  
xlabel('Temps (s)');  
ylabel('Amplitude');  
title('Signal reconstruit x_{rec}[n] (déconvolution naïve)');  
grid on;
```

qst a.12

```
[x_rec_bruite, reste_bruite] = deconv(y_quant, h);  
  
% Base de temps pour le signal reconstruit  
tx_rec_bruite = (0:length(x_rec_bruite)-1) * Te;  
  
% Représentation graphique du signal reconstruit  
figure;  
plot(tx_rec_bruite, x_rec_bruite, 'LineWidth', 1.5);  
xlabel('Temps (s)');  
ylabel('Amplitude');  
title('Signal reconstruit  $x_{\text{rec}}[n]$  (signal bruité)');  
grid on;
```



qst a.13

```
% Initialisation des paramètres
sigma_values = [2, 4, 8]; % Différentes valeurs de sigma
Nh = 30; % Horizon temporel du filtre
reconstruction_errors = zeros(1, length(sigma_values)); % Erreurs de
reconstruction

figure;
for i = 1:length(sigma_values)
    sigma = sigma_values(i); % Changer la valeur de sigma

    % Recalcul de la réponse impulsionnelle h[n]
    n = 0:Nh-1; % Indices temporels
    t = n * Te; % Base de temps
    h = (1 / (sigma * sqrt(2 * pi))) * exp(-((t - m).^2) / (2 * sigma^2));

    % Déconvolution "naïve" avec y bruité
    [x_rec, ~] = deconv(y_quant, h);

    % Base de temps pour x_rec
    tx_rec = (0:length(x_rec)-1) * Te;

    % Calcul de l'erreur de reconstruction
    if length(x_rec) > length(x)
```

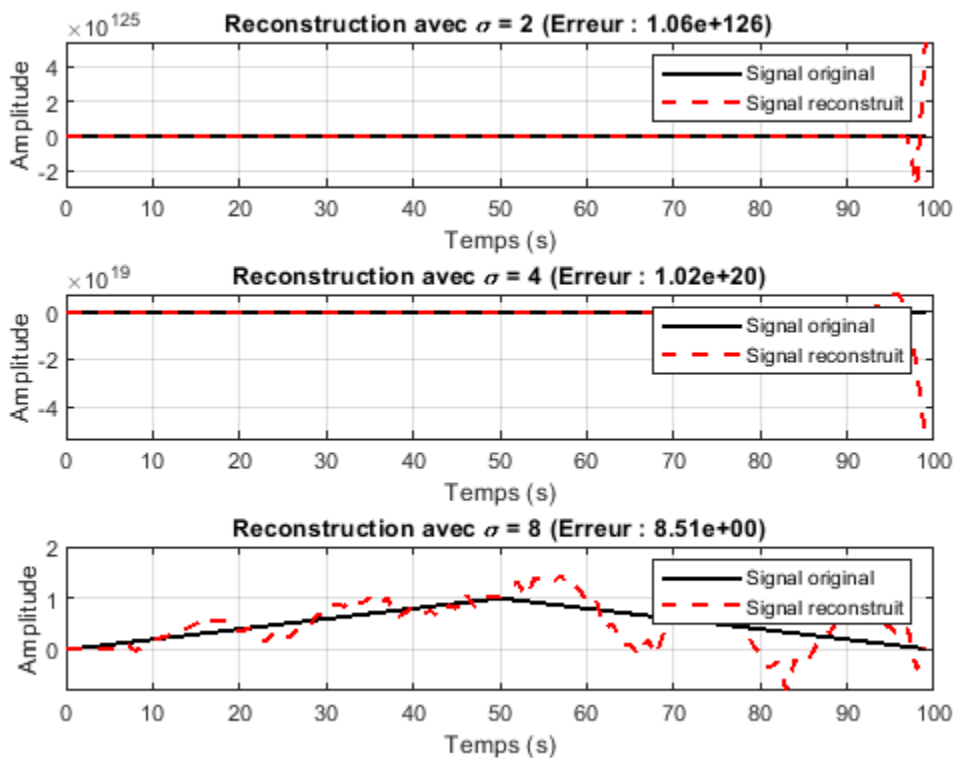
```

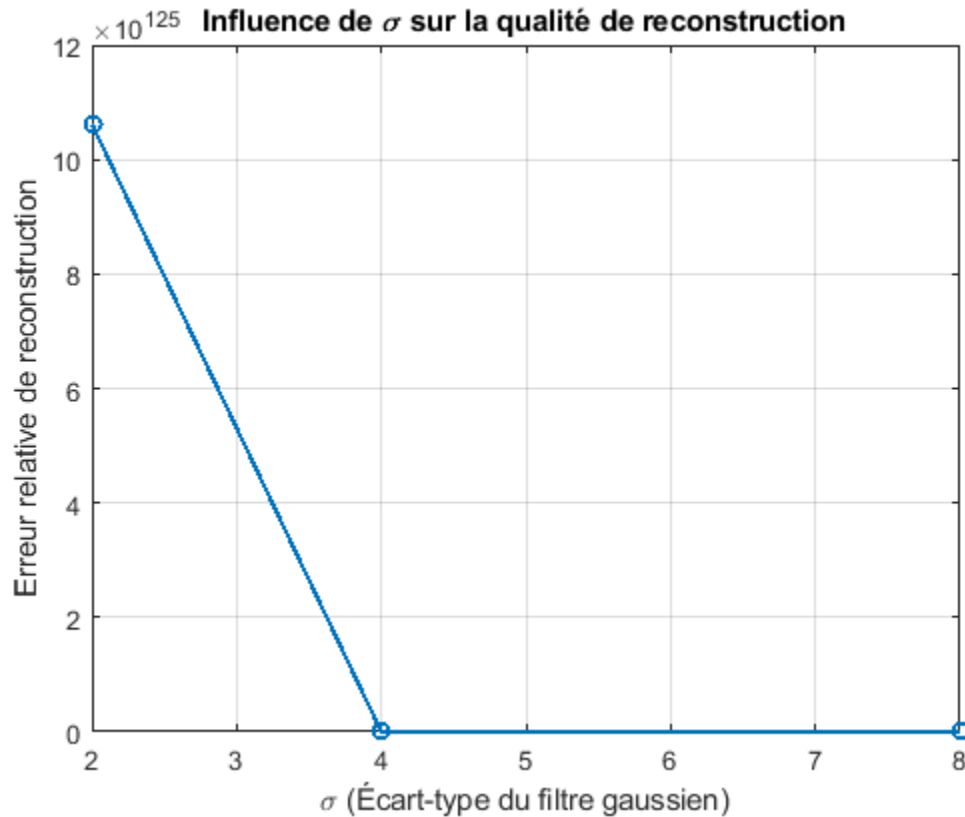
    x_rec = x_rec(1:length(x)); % Tronquer si nécessaire
end
reconstruction_error = norm(x - x_rec) / norm(x); % Erreur relative
reconstruction_errors(i) = reconstruction_error;

% Tracé du signal reconstruit
subplot(length(sigma_values), 1, i);
plot(tx, x, 'k', 'LineWidth', 1.5); hold on; % Signal original
plot(tx_rec, x_rec, 'r--', 'LineWidth', 1.5); % Signal reconstruit
xlabel('Temps (s)');
ylabel('Amplitude');
title(['Reconstruction avec \sigma = ', num2str(sigma), ...
      ' (Erreur : ', num2str(reconstruction_error, '%.2e'), ')']);
legend('Signal original', 'Signal reconstruit');
grid on;
end

% Affichage des erreurs de reconstruction
figure;
plot(sigma_values, reconstruction_errors, '-o', 'LineWidth', 1.5);
xlabel('\sigma (Écart-type du filtre gaussien)');
ylabel('Erreur relative de reconstruction');
title('Influence de \sigma sur la qualité de reconstruction');
grid on;

```





qst b.14

```
% Convertir les vecteurs en colonnes
h = h(:);
y_quant = y_quant(:);

% Construction de la matrice de convolution H
prem_col = [h; zeros(length(y_quant) - length(h), 1)];
prem_lig = [h(1); zeros(length(x) - 1, 1)];
H = toeplitz(prem_col, prem_lig);

% Vérification des dimensions
disp(['Dimensions de H : ', mat2str(size(H))]);
disp(['Dimensions de y_quant : ', mat2str(size(y_quant))]);

% Résolution par moindres carrés
x_rec_ls = (H' * H) \ (H' * y_quant);

% Affichage
tx_rec_ls = (0:length(x_rec_ls)-1) * Te;
figure;
plot(tx, x, 'k', 'LineWidth', 1.5); hold on;
plot(tx_rec_ls, x_rec_ls, 'r--', 'LineWidth', 1.5);
xlabel('Temps (s)');
ylabel('Amplitude');
```

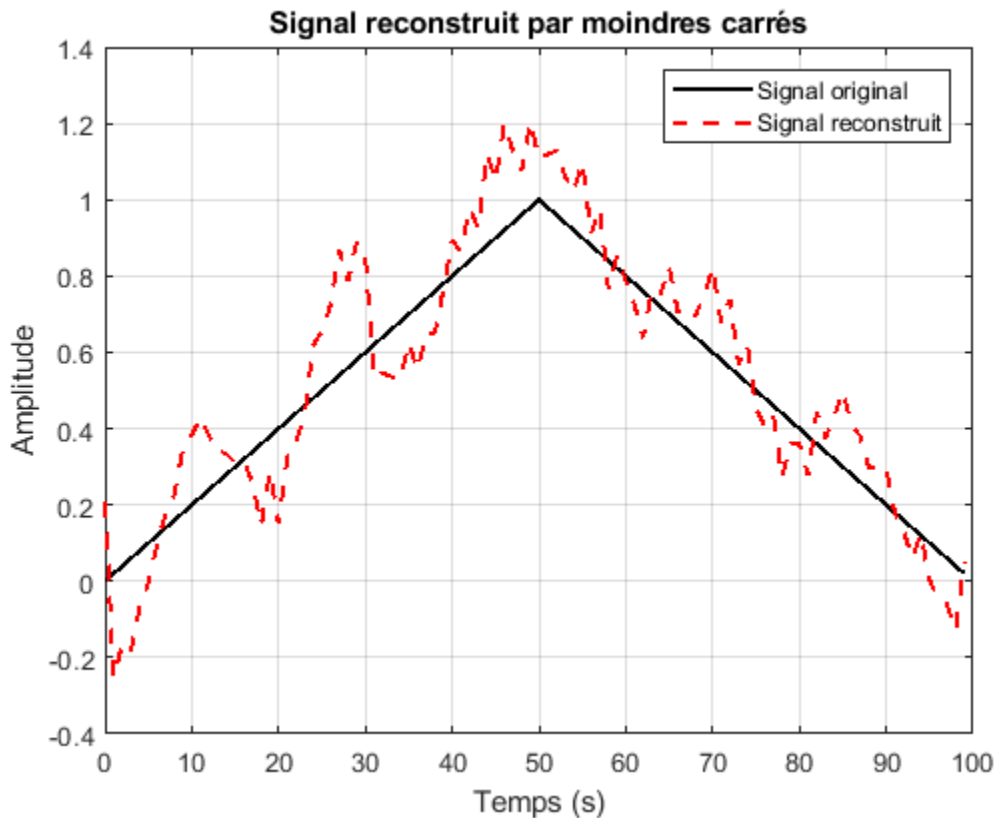
```

title('Signal reconstruit par moindres carrés');
legend('Signal original', 'Signal reconstruit');
grid on;

```

Dimensions de H : [129 100]

Dimensions de y_{quant} : [129 1]



qst b.15

```

% Tracé du signal original et reconstruit
figure;
plot(tx, x, 'k', 'LineWidth', 1.5); hold on; % Signal original
plot(tx_rec_ls, x_rec_ls, 'r--', 'LineWidth', 1.5); % Signal reconstruit
xlabel('Temps (s)');
ylabel('Amplitude');
title('Signal original et reconstruit par moindres carrés');
legend('Signal original', 'Signal reconstruit');
grid on;

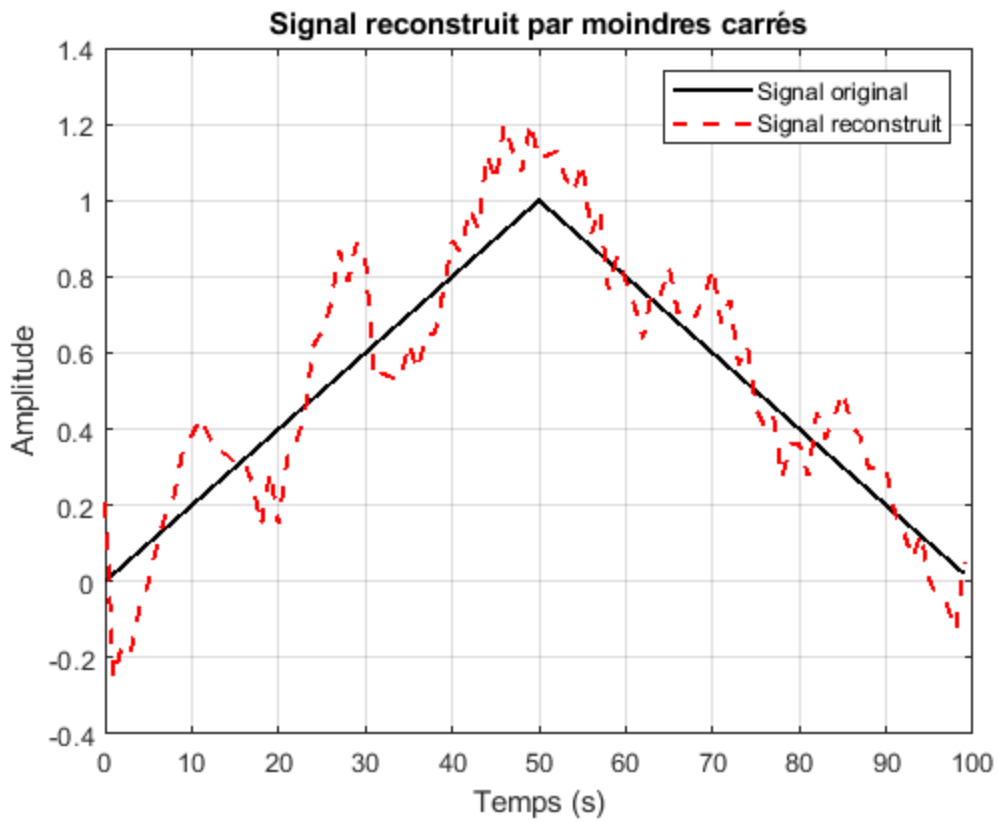
% Calcul de l'énergie de l'erreur de reconstruction
if length(x_rec_ls) > length(x)
    x_rec_ls = x_rec_ls(1:length(x)); % Tronquer si nécessaire
end
error_signal = x - x_rec_ls; % Signal d'erreur
error_energy = sum(error_signal.^2); % Énergie de l'erreur

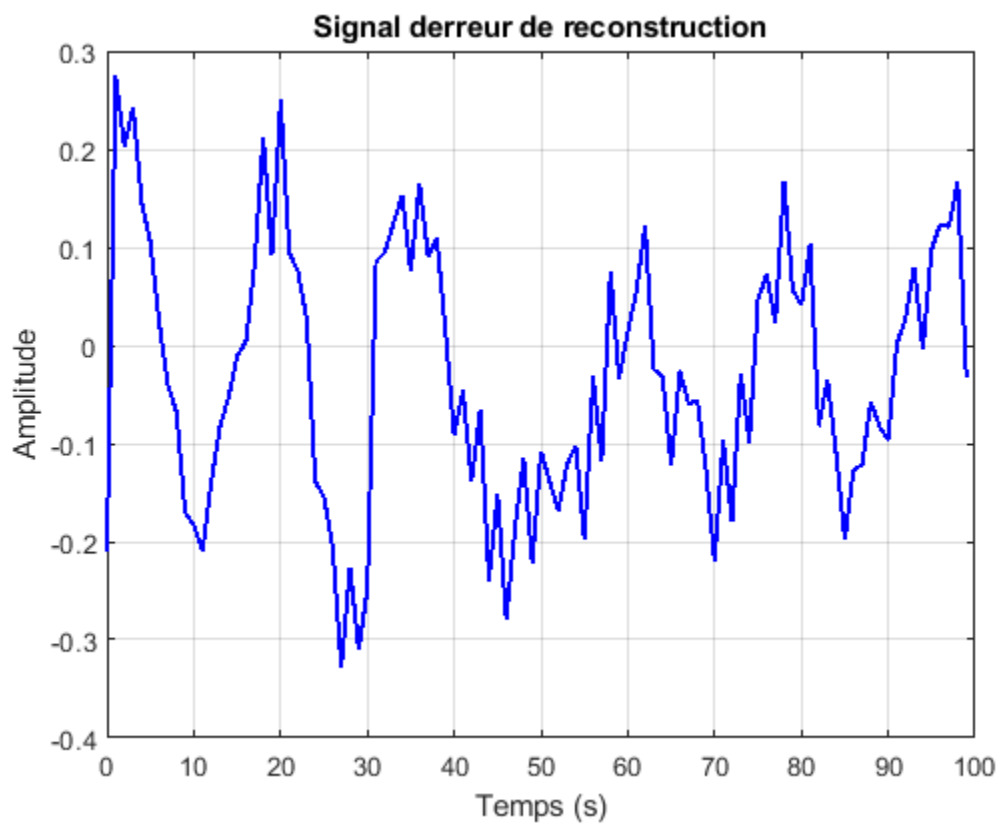
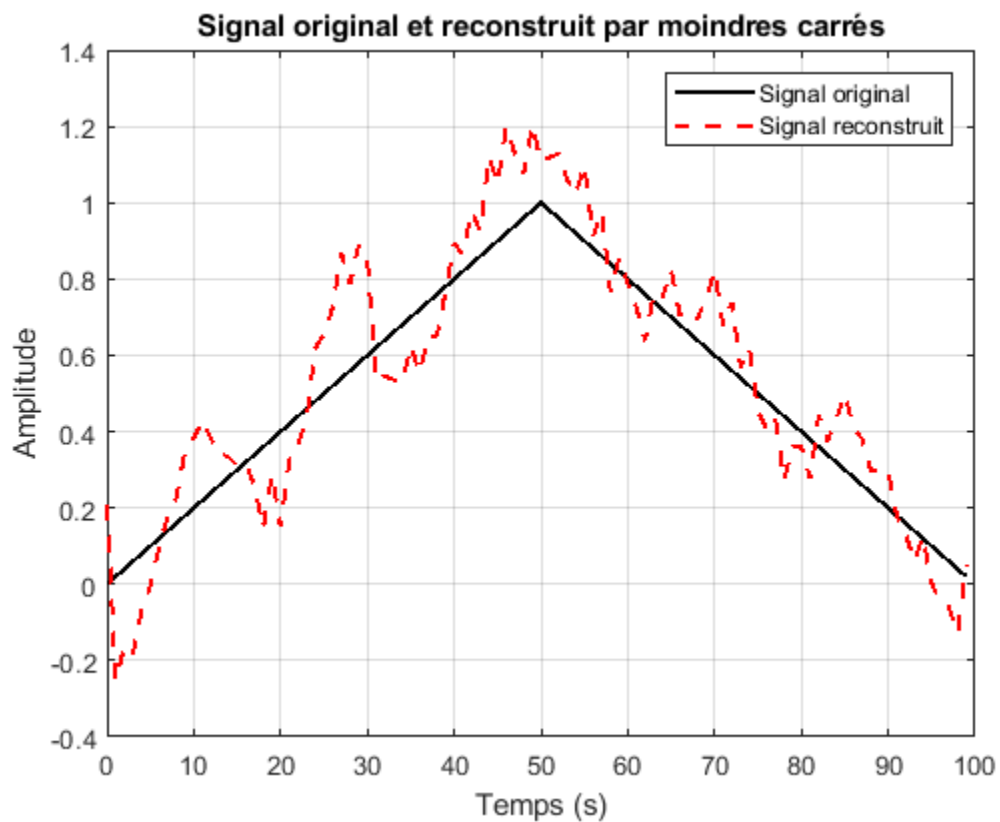
```

```
% Affichage du résultat
fprintf('Énergie de l'erreur de reconstruction : %.5e\n', error_energy);

% Tracé du signal d'erreur
figure;
plot(tx, error_signal, 'b', 'LineWidth', 1.5);
xlabel('Temps (s)');
ylabel('Amplitude');
title('Signal d'erreur de reconstruction');
grid on;
```

Énergie de l'erreur de reconstruction : 1.87596e+00





qst b.16

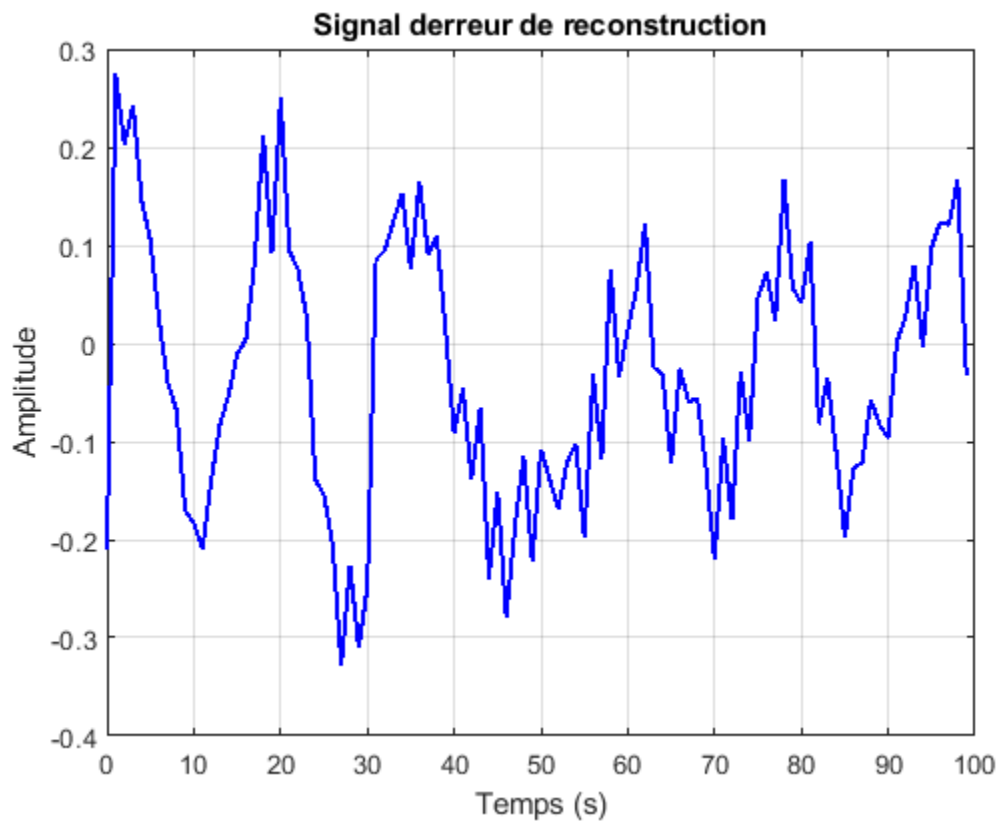
```
% Calcul des valeurs singulières de la matrice de convolution H
singular_values = svd(H);

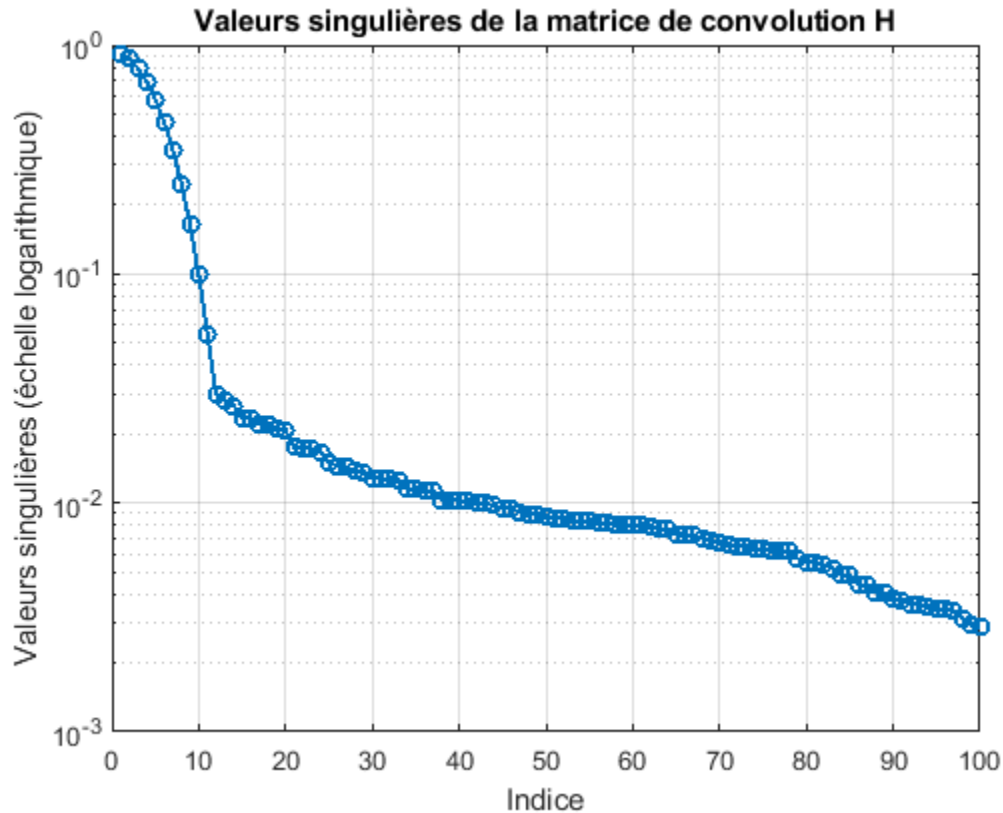
% Calcul du conditionnement : ratio entre la plus grande et la plus petite
valeur singulière non nulle
condition_number = max(singular_values) /
min(singular_values(singular_values > eps));

% Affichage du conditionnement
fprintf('Conditionnement de H : %.2e\n', condition_number);

% Tracé des valeurs singulières
figure;
semilogy(singular_values, 'o-', 'LineWidth', 1.5);
xlabel('Indice');
ylabel('Valeurs singulières (échelle logarithmique)');
title('Valeurs singulières de la matrice de convolution H');
grid on;
```

Conditionnement de H : 3.22e+02





qst b.17

```
% Initialisation des paramètres
sigma_values = [2, 4, 8, 12, 16]; % Différentes valeurs de sigma
condition_numbers = zeros(1, length(sigma_values)); % Stocker les
conditionnements

% Parcours des différentes valeurs de sigma
for i = 1:length(sigma_values)
    sigma = sigma_values(i);

    % Calcul de la réponse impulsionnelle h[n] pour chaque sigma
    Nh = 30; % Horizon temporel du filtre
    n = 0:Nh-1;
    t = n * Te;
    h = (1 / (sigma * sqrt(2 * pi))) * exp(-((t - m).^2) / (2 * sigma^2));

    % Construction de la matrice de convolution H
    prem_col = [h(:); zeros(length(x) - length(h), 1)];
    prem_lig = [h(1); zeros(length(x) - 1, 1)];
    H = toeplitz(prem_col, prem_lig);

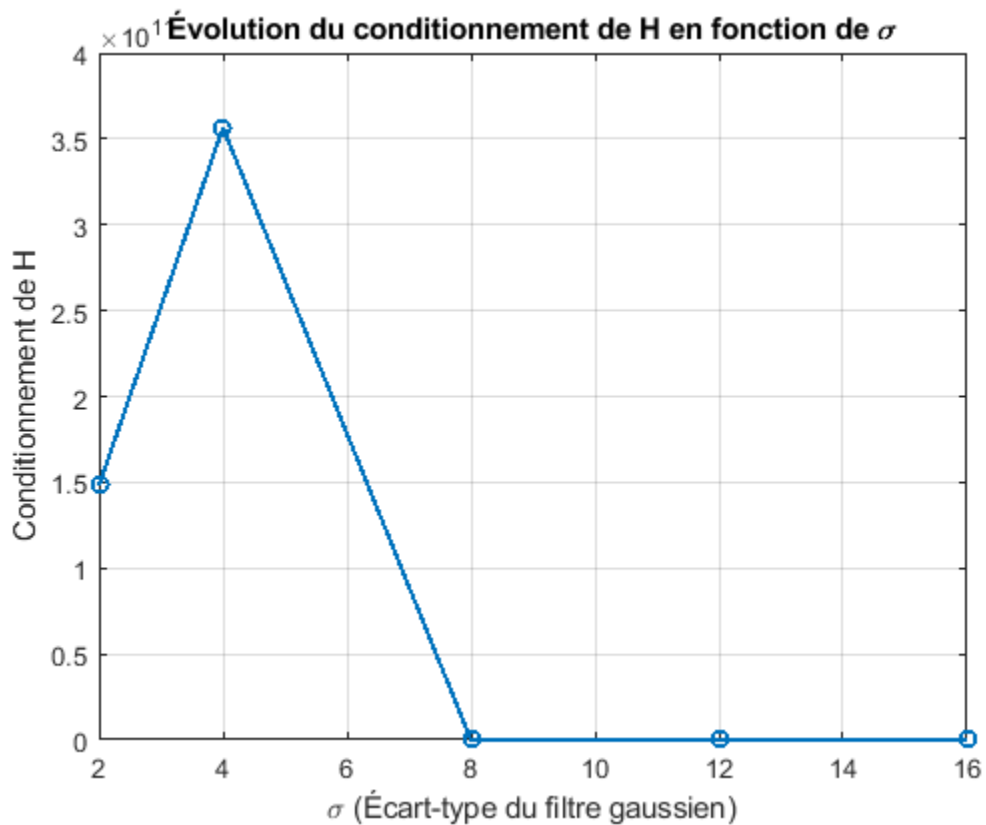
    % Calcul des valeurs singulières
    singular_values = svd(H);
```

```

    % Conditionnement de H
    condition_numbers(i) = max(singular_values) /
min(singular_values(singular_values > eps));
end

% Affichage des conditionnements en fonction de sigma
figure;
plot(sigma_values, condition_numbers, '-o', 'LineWidth', 1.5);
xlabel('\sigma (Écart-type du filtre gaussien)');
ylabel('Conditionnement de H');
title('Évolution du conditionnement de H en fonction de \sigma');
grid on;

```



qst c.18

```

% Vérification que x est un vecteur colonne
x = x(:); % Convertir en colonne si nécessaire

% Initialisation de la matrice D1
Nx = length(x); % Dimension de la matrice
dcol1 = zeros(1, Nx);
dcol1(1:2) = [1 -1]; % Valeurs pour les différences premières
dlig1 = zeros(1, Nx);
dlig1(1) = dcol1(1);

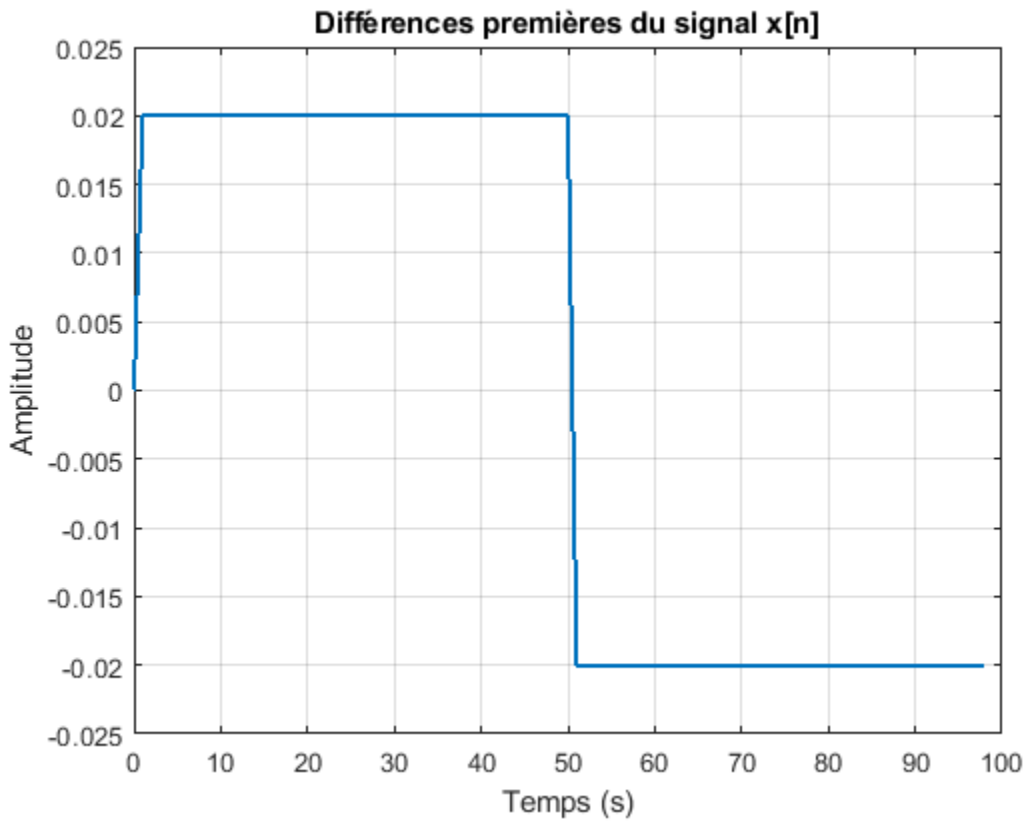
% Construction de la matrice D1 à l'aide de la fonction toeplitz

```

```
D1 = toeplitz(dcoll, dlig1);

% Calcul des différences premières
diff_premieres = D1 * x;

% Affichage du signal des différences premières
figure;
plot(tx(1:end-1), diff_premieres(1:end-1), 'LineWidth', 1.5);
xlabel('Temps (s)');
ylabel('Amplitude');
title('Différences premières du signal x[n]');
grid on;
```



Published with MATLAB® R2024a